

PocketSmart AI: Your Smart Budget

Login and Register Page Routings

NAAN MUDHALVAN PROJECT

20-Page Detailed Authentication Routing Report

A complete project document covering registration, login, authentication, sessions, protected routes, logout, validation, testing, security, documentation, and demonstration.

1. Introduction — Login and Register Page Routings

PocketSmart AI: Your Smart Budget is the main title of this Naan Mudhalvan project. The subtitle of this document is “Login and Register Page Routings.” This part of the project explains how a user enters the application, creates an account, verifies identity when required, logs in, receives an authenticated session, and reaches the protected budgeting dashboard. The routing is designed to be secure, understandable, testable, and easy to demonstrate. Registration creates a user profile, while login verifies the credentials of an existing user. The routing should keep authentication logic separate from budgeting calculations so that security and financial functionality remain modular.

- Main title: PocketSmart AI: Your Smart Budget.
- Project context: Naan Mudhalvan.
- Technical subtitle: Login and Register Page Routings.
- Registration creates a new user account.
- Login verifies an existing account.
- Validation checks all submitted fields.
- Password values should never be stored as plain text.
- Authentication should create a secure session or token.
- Protected pages should require authentication.
- Logout should invalidate the active session.
- Forgot-password flow can be added as an optional module.
- Email or phone verification can be added if required.
- Duplicate-account checks should be performed.
- Invalid credentials should produce safe error messages.
- Authentication failures should not reveal sensitive account information.
- Rate limiting can reduce repeated automated login attempts.
- Database access should be separated from UI code.
- Authentication results should be returned in structured form.
- Successful login should redirect to the dashboard.
- Successful registration can redirect to login or directly to onboarding.
- The budgeting application should not be accessible through protected routes without authorization.
- The same authentication service can support web or mobile interfaces.
- Testing must cover successful and unsuccessful flows.
- Documentation should describe every authentication route.
- This report explains the complete login and registration routing flow.

Routing principle: User action → UI route → validation → authentication service → data layer → authenticated state → protected application route. Each step should have defined inputs, outputs, validation, and test evidence.

Security principle: Authentication and authorization must be enforced on the server or trusted application layer; hiding a page in the interface alone is not sufficient to protect user data.

2. Objectives of Login and Registration

The authentication module provides the entry and identity-management foundation of PocketSmart AI. Its objective is not simply to create two screens; it must connect user interface actions to validation, database operations, password security, session management, and protected application routes. The authentication system should be simple enough for users while following appropriate security practices. The project can implement only the essential functions first and add advanced account recovery and verification later.

- Objective 1: provide a registration page.
- Objective 2: provide a login page.
- Objective 3: validate registration fields.
- Objective 4: validate login fields.
- Objective 5: prevent duplicate accounts.
- Objective 6: securely store passwords using a suitable password-hashing mechanism.
- Objective 7: authenticate valid credentials.
- Objective 8: reject invalid credentials.
- Objective 9: create an authenticated session or token.
- Objective 10: protect the budget dashboard.
- Objective 11: support logout.
- Objective 12: provide safe error messages.
- Objective 13: support future password recovery.
- Objective 14: support optional verification.
- Objective 15: record necessary security events without exposing passwords.
- Objective 16: provide testable routing.
- Objective 17: provide documentation for evaluators.
- Objective 18: make the live demonstration clear.
- The login system becomes the gateway to PocketSmart AI.
- It establishes user identity before saved budgets are accessed.
- It supports separation between public and protected pages.
- It provides a foundation for personal budget persistence.
- The design should be extensible.
- The final authentication workflow should be reliable and easy to explain.

Routing principle: User action → UI route → validation → authentication service → data layer → authenticated state → protected application route. Each step should have defined inputs, outputs, validation, and test evidence.

Security principle: Authentication and authorization must be enforced on the server or trusted application layer; hiding a page in the interface alone is not sufficient to protect user data.

3. Authentication Architecture

The login and registration architecture should use clear layers. The frontend presents forms and sends requests. A route/controller receives the request and passes it to a validation service. The authentication service checks the data, interacts with the user repository, and uses a password-hashing component for password verification or storage. On successful login, a session or token service creates authenticated state. The authorization middleware then protects budget routes. This layered architecture keeps credentials away from unrelated budgeting code.

- Frontend layer: Register and Login pages.
- Routing layer: registration and login endpoints.
- Validation layer: field and format validation.
- Authentication service: registration and credential verification.
- Password service: secure password hashing and comparison.
- User repository: database operations.
- Session/token service: authenticated state.
- Authorization middleware: protects private routes.
- Logout service: terminates authentication state.
- Error handler: produces safe responses.
- Audit/security logging: records important security events.
- Database: stores user profile and password hash.
- Budget service: remains separate from authentication.
- Dashboard: consumes authenticated user identity.
- Registration route creates a user.
- Login route verifies a user.
- Protected route checks session/token.
- Logout route removes or invalidates authentication state.
- The architecture can be implemented in common web frameworks.
- The same conceptual design can support mobile APIs.
- Unit tests target services.
- Integration tests target routes.
- End-to-end tests target complete user journeys.
- The architecture is suitable for an academic project demonstration.
- It also creates a foundation for future production hardening.

Routing principle: User action → UI route → validation → authentication service → data layer → authenticated state → protected application route. Each step should have defined inputs, outputs, validation, and test evidence.

Security principle: Authentication and authorization must be enforced on the server or trusted application layer; hiding a page in the interface alone is not sufficient to protect user data.

4. Register Page — UI Routing

The Register page is the first authentication route for a new user. It should provide a simple form with fields appropriate to the project's requirements. A typical form may contain name, email or username, password, and confirm password. Additional fields should be included only when required. The page should explain password requirements clearly and should not expose sensitive technical information. When the user submits the form, the browser sends a structured request to the registration route.

- Page route example: /register.
- Display application logo/title.
- Display PocketSmart AI name.
- Display registration heading.
- Display name field when required.
- Display email or username field.
- Display password field.
- Display confirm-password field.
- Display password requirement guidance.
- Display Register button.
- Display link to Login page.
- Client-side validation can provide immediate feedback.
- Server-side validation remains mandatory.
- Empty required fields should be rejected.
- Invalid email format should be rejected when email is required.
- Password mismatch should be rejected.
- Weak passwords should be rejected according to defined rules.
- Successful submission sends data to the authentication route.
- Sensitive values should be transmitted securely in production.
- Password should never be printed in console logs.
- Registration result should be displayed clearly.
- Successful registration can redirect to login.
- Optional onboarding can follow successful registration.
- The Register page is the starting point of the new-user routing.
- The page should remain visually simple for a strong demonstration.

Routing principle: User action → UI route → validation → authentication service → data layer → authenticated state → protected application route. Each step should have defined inputs, outputs, validation, and test evidence.

Security principle: Authentication and authorization must be enforced on the server or trusted application layer; hiding a page in the interface alone is not sufficient to protect user data.

5. Register API Routing

The registration API receives the form data and performs the server-side workflow. The route should first validate the request. It then checks whether an account already exists. If no duplicate is found, the password is transformed into a secure password hash before storage. The service creates the user record and returns a safe success response. The plain password should never be stored or returned.

- Route example: POST /api/auth/register.
- Request: name.
- Request: email or username.
- Request: password.
- Request: confirmPassword when validation is performed server-side.
- Validate request structure.
- Validate required fields.
- Validate email or username format.
- Validate password rules.
- Compare password and confirmation.
- Query user repository for duplicate account.
- If duplicate exists, return a safe conflict response.
- If valid, generate a password hash.
- Create user record.
- Store only the required profile information.
- Return success response without password.
- Optionally create initial user preferences.
- Optionally create an empty budget profile.
- Do not automatically expose protected budget information.
- Return a route-safe response.
- Frontend displays success or error.
- Integration tests verify repository interaction.
- Security tests verify that plaintext passwords are not persisted.
- Registration routing is complete after a safe account is created.
- The route can later support email verification.

Routing principle: User action → UI route → validation → authentication service → data layer → authenticated state → protected application route. Each step should have defined inputs, outputs, validation, and test evidence.

Security principle: Authentication and authorization must be enforced on the server or trusted application layer; hiding a page in the interface alone is not sufficient to protect user data.

6. Login Page — UI Routing

The Login page allows an existing user to access the PocketSmart AI dashboard. It should be minimal and focused. A user typically enters email or username and password. The page sends these values to the login endpoint. The response determines whether the user can proceed. The interface should not reveal whether a specific account exists through overly detailed error messages, because account enumeration can create a security risk.

- Page route example: /login.
- Display PocketSmart AI title.
- Display login heading.
- Display email or username field.
- Display password field.
- Display Login button.
- Display Register link.
- Optional Forgot Password link.
- Validate required fields.
- Do not display password in plain text by default.
- Use password visibility control only if appropriate.
- Send login request to secure authentication endpoint.
- Disable repeated submission while request is processing.
- Display safe authentication error.
- Successful login redirects to dashboard.
- Failed login keeps the user on the login page.
- Clear sensitive form state when appropriate.
- Do not expose server stack traces.
- Do not show database errors to the user.
- Provide accessible labels.
- Provide keyboard-friendly navigation.
- Use responsive layout.
- Login page should be easy to demonstrate live.
- The page is the public entry point for returning users.
- It connects directly to the login API route.

Routing principle: User action → UI route → validation → authentication service → data layer → authenticated state → protected application route. Each step should have defined inputs, outputs, validation, and test evidence.

Security principle: Authentication and authorization must be enforced on the server or trusted application layer; hiding a page in the interface alone is not sufficient to protect user data.

7. Login API Routing

The login API verifies submitted credentials. The service retrieves the user record using the identifier, compares the submitted password with the stored password hash, and creates authenticated state when verification succeeds. The response should contain only the information necessary for the frontend. Tokens or session identifiers should be handled securely according to the chosen architecture.

- Route example: POST /api/auth/login.
- Request: email or username.
- Request: password.
- Validate request.
- Find user record.
- If no matching user is found, return a generic authentication failure.
- If user exists, compare password with stored hash.
- If comparison fails, return generic authentication failure.
- If comparison succeeds, create authenticated session or token.
- Return safe user information.
- Never return password hash to frontend.
- Do not return plaintext password.
- Set secure cookie when cookie-based sessions are used.
- Apply appropriate cookie security attributes in production.
- Use token expiration when token-based authentication is used.
- Record security event when appropriate.
- Apply rate limiting where supported.
- Redirect authenticated user to dashboard.
- Protected API requests carry authentication state.
- Authorization middleware checks the authenticated identity.
- Login service remains separate from budget calculations.
- Tests verify success and failure paths.
- Tests verify protected-route behavior.
- Login routing is the central authentication flow for returning users.

Routing principle: User action → UI route → validation → authentication service → data layer → authenticated state → protected application route. Each step should have defined inputs, outputs, validation, and test evidence.

Security principle: Authentication and authorization must be enforced on the server or trusted application layer; hiding a page in the interface alone is not sufficient to protect user data.

8. Password Security Routing

Password handling is one of the most important parts of authentication. The application should use a modern password-hashing approach rather than storing passwords directly. Password verification compares the entered password with the stored hash through the chosen password library. The exact algorithm depends on the implementation stack and security requirements. The key routing principle is that plaintext passwords exist only during the controlled authentication operation and are never stored in the database.

- Registration receives plaintext password through a secure request.
- Validation checks password requirements.
- Password hashing function creates a password hash.
- Database stores the hash, not the original password.
- Login receives the submitted password.
- User repository retrieves the stored hash.
- Password comparison function checks the password.
- Successful comparison allows authentication.
- Failed comparison denies authentication.
- Hash should not be returned to the frontend.
- Password should not be written to logs.
- Password should not appear in error messages.
- Database backups should protect credential data.
- Access to authentication tables should be restricted.
- Password reset should use a separate secure workflow.
- Password changes should require authentication or secure recovery.
- Security testing should inspect storage behavior.
- Code review should check for accidental password logging.
- Transport security should protect credentials in production.
- Rate limiting can reduce repeated guessing attempts.
- Account lockout policies should be designed carefully to avoid denial-of-service abuse.
- Authentication errors should remain generic.
- The password route is separate from budgeting calculations.
- Secure credential handling is essential for the project.

Routing principle: User action → UI route → validation → authentication service → data layer → authenticated state → protected application route. Each step should have defined inputs, outputs, validation, and test evidence.

Security principle: Authentication and authorization must be enforced on the server or trusted application layer; hiding a page in the interface alone is not sufficient to protect user data.

9. Database User Routing

The user database layer stores account information needed for authentication. A basic user table can contain an identifier, name, login identifier, password hash, account status, creation timestamp, and update timestamp. The exact fields depend on project requirements. The data-access layer should provide functions for creating users, finding users, updating users, and disabling accounts when required. It should not contain UI logic.

- User ID should be unique.
- Email or username should be unique when used as login identifier.
- Password hash should be stored securely.
- Name can be stored when required.
- Account status can indicate active or disabled.
- Created timestamp records account creation.
- Updated timestamp records profile changes.
- Optional verification status can be stored.
- Optional last-login timestamp can be stored.
- Repository function: createUser().
- Repository function: findUserByIdentifier().
- Repository function: findUserById().
- Repository function: updateUser().
- Repository function: disableUser().
- Authentication service calls repository functions.
- Routes should not contain raw database logic everywhere.
- Database errors should be handled safely.
- Sensitive database access should be restricted.
- Backups should be protected.
- Test database should be separated from production data.
- Seed users can support demonstration testing.
- Database constraints should prevent duplicate identifiers.
- User data should be minimized.
- Authentication data should not be mixed unnecessarily with budget transaction data.
- This routing makes the authentication module maintainable.

Routing principle: User action → UI route → validation → authentication service → data layer → authenticated state → protected application route. Each step should have defined inputs, outputs, validation, and test evidence.

Security principle: Authentication and authorization must be enforced on the server or trusted application layer; hiding a page in the interface alone is not sufficient to protect user data.

10. Session and Token Routing

After successful login, the application needs a way to remember that the user has been authenticated. Two common approaches are server-managed sessions or signed tokens. The project can choose one based on its technology stack. The important routing is that protected requests carry authenticated state, and the server verifies that state before returning private budget information. Authentication state should have a defined lifetime and logout behavior.

- Login success triggers session/token creation.
- Session approach stores authentication state server-side.
- Token approach sends a signed token with protected requests.
- Authentication state identifies the user.
- Expiration limits long-lived unauthorized access.
- Protected request reaches authentication middleware.
- Middleware verifies session or token.
- Valid state allows request to continue.
- Invalid state returns unauthorized response.
- Expired state requires login again.
- Logout invalidates session or token as appropriate.
- Cookie-based sessions require appropriate security attributes.
- Token-based systems require secure token storage strategy.
- Do not place unnecessary sensitive information in tokens.
- Do not trust client-provided user identifiers without verification.
- Budget ownership should be checked after authentication.
- One user should not access another user's budget.
- Tests should cover missing authentication.
- Tests should cover expired authentication.
- Tests should cover invalid authentication.
- Tests should cover successful authentication.
- The session/token layer connects login with all protected PocketSmart AI features.
- It is the bridge between identity and authorization.
- This makes the dashboard accessible only to authenticated users.

Routing principle: User action → UI route → validation → authentication service → data layer → authenticated state → protected application route. Each step should have defined inputs, outputs, validation, and test evidence.

Security principle: Authentication and authorization must be enforced on the server or trusted application layer; hiding a page in the interface alone is not sufficient to protect user data.

11. Protected Dashboard Routing

The dashboard is a protected page because it may display saved budgets and financial information. When a user attempts to open it, the application checks authentication before loading the page or its data. The route then loads only budgets belonging to the authenticated user. This creates two security layers: authentication establishes who the user is, while authorization determines what that user can access.

- Page route example: /dashboard.
- Authentication middleware checks session/token.
- Unauthenticated request is redirected to login or receives unauthorized response.
- Authenticated request continues.
- User ID is obtained from trusted authentication state.
- Budget service queries budgets for that user.
- Dashboard displays permitted budgets.
- User can create a new budget.
- User can open an existing budget.
- User can edit permitted budget information.
- User cannot access another user's budget by changing an ID in the URL.
- Server-side ownership checks are required.
- UI hiding alone is not sufficient authorization.
- API routes should also verify ownership.
- Dashboard can display profile information.
- Dashboard can display budget summaries.
- Dashboard can display savings goals.
- Dashboard can display category analysis.
- Dashboard can display AI explanations.
- Logout control should be visible.
- Authentication errors should be handled safely.
- End-to-end testing should verify the protected flow.
- This routing connects authentication with the main PocketSmart AI functionality.
- It is one of the most important demonstrations for the project.

Routing principle: User action → UI route → validation → authentication service → data layer → authenticated state → protected application route. Each step should have defined inputs, outputs, validation, and test evidence.

Security principle: Authentication and authorization must be enforced on the server or trusted application layer; hiding a page in the interface alone is not sufficient to protect user data.

12. Logout Routing

Logout ends the authenticated session. The route depends on the authentication architecture. For a server-side session, the session can be destroyed or invalidated. For a token-based design, the application can clear the client-side authentication state and use expiration or a server-side revocation strategy when appropriate. After logout, protected pages should no longer be accessible using the previous authentication state.

- Route example: POST /api/auth/logout.
- User selects Logout.
- Frontend sends logout request.
- Server validates current authentication where required.
- Session is invalidated for session-based authentication.
- Client authentication state is cleared.
- Secure cookie can be cleared.
- Token is removed from approved client storage.
- Optional token revocation can be implemented.
- User is redirected to Login.
- Protected API call after logout should fail.
- Back-button behavior should not expose newly fetched private data.
- Cached private data should be handled carefully.
- Logout should not delete the user's budgets.
- Logout only ends the authentication state.
- Tests should verify protected-route rejection after logout.
- Tests should verify repeated logout handling.
- UI should provide clear confirmation when necessary.
- Security events can be recorded without storing credentials.
- Logout should be available from the dashboard.
- The route closes the standard authentication lifecycle.
- It completes the register/login/logout demonstration.
- It also prepares the application for another user session.

Routing principle: User action → UI route → validation → authentication service → data layer → authenticated state → protected application route. Each step should have defined inputs, outputs, validation, and test evidence.

Security principle: Authentication and authorization must be enforced on the server or trusted application layer; hiding a page in the interface alone is not sufficient to protect user data.

13. Forgot Password and Account Recovery Routing

Forgot-password functionality is optional but valuable for a complete authentication system. A secure recovery workflow should not reveal whether an account exists. The user submits an identifier, and the system can send a time-limited recovery mechanism through a verified channel when such infrastructure is available. The recovery process must never expose the existing password because passwords are stored as hashes.

- Page route example: /forgot-password.
- User enters email or username.
- Server validates request.
- Server returns a generic response.
- If account exists, a secure recovery process may begin.
- Generate a short-lived recovery token.
- Do not place passwords inside recovery links.
- Send recovery instructions through a verified channel when implemented.
- Recovery token should expire.
- Recovery token should be single-use where appropriate.
- Reset page accepts the valid recovery token.
- User creates a new password.
- New password is validated.
- New password is securely hashed.
- Old password hash is replaced.
- Existing sessions can be invalidated depending on security policy.
- User is directed to login.
- Recovery errors should be generic.
- Test expired recovery token.
- Test invalid recovery token.
- Test successful password reset.
- Test password mismatch.
- Test weak password.
- Account recovery should be documented as optional scope if not implemented.
- This routing demonstrates awareness of complete authentication lifecycle design.

Routing principle: User action → UI route → validation → authentication service → data layer → authenticated state → protected application route. Each step should have defined inputs, outputs, validation, and test evidence.

Security principle: Authentication and authorization must be enforced on the server or trusted application layer; hiding a page in the interface alone is not sufficient to protect user data.

14. Email or Account Verification Routing

Account verification can be added when the project requires confirmed email ownership. Registration creates the account in a pending state. A verification mechanism is generated and sent to the user's registered address. The user follows the verification flow, after which the account becomes active. If the project does not have email infrastructure, the report should clearly mark this feature as optional rather than pretending that verification is implemented.

- Registration creates pending account when verification is enabled.
- Generate verification token.
- Set token expiration.
- Send verification message through configured service.
- User opens verification route.
- Server validates token.
- Expired token is rejected.
- Invalid token is rejected.
- Valid token activates account.
- Verification status is stored.
- Login can require verified status if project policy requires it.
- Resend verification can be an optional route.
- Resend attempts should be rate limited.
- Do not expose unnecessary account information.
- Tests should cover valid token.
- Tests should cover expired token.
- Tests should cover invalid token.
- Tests should cover repeated use.
- Testing should verify account state transitions.
- Documentation should identify whether this feature is implemented or future scope.
- The verification route adds an extra trust step.
- It should not be included in the live demonstration unless configured.
- The basic login/register workflow remains functional without it.
- Verification can be integrated later.
- This provides a scalable authentication roadmap.

Routing principle: User action → UI route → validation → authentication service → data layer → authenticated state → protected application route. Each step should have defined inputs, outputs, validation, and test evidence.

Security principle: Authentication and authorization must be enforced on the server or trusted application layer; hiding a page in the interface alone is not sufficient to protect user data.

15. Validation and Error Handling

Validation is essential in both registration and login. Client-side validation improves usability, but server-side validation is required for security and correctness. Error responses should help the user correct a problem without exposing internal information. The project should define validation rules before coding so that UI behavior, backend behavior, and test cases remain consistent.

- Registration: required name when applicable.
- Registration: required email/username.
- Registration: valid identifier format.
- Registration: password meets defined policy.
- Registration: confirmation matches password.
- Registration: duplicate identifier rejected.
- Login: required identifier.
- Login: required password.
- Login: invalid credentials handled safely.
- Forgot password: valid input format.
- Verification: token format and expiration.
- Protected route: authentication required.
- Protected route: ownership required.
- Database error: safe generic message.
- Network error: retry guidance.
- Rate-limit response: safe message.
- Never display stack traces to ordinary users.
- Never display password hashes.
- Never echo submitted passwords in errors.
- Do not reveal whether a login identifier exists when policy requires generic errors.
- Validation should be consistent.
- Validation rules should be documented.
- Validation cases should be tested.
- Error routing is part of the functionality, not an afterthought.
- A strong validation design improves both security and user experience.

Routing principle: User action → UI route → validation → authentication service → data layer → authenticated state → protected application route. Each step should have defined inputs, outputs, validation, and test evidence.

Security principle: Authentication and authorization must be enforced on the server or trusted application layer; hiding a page in the interface alone is not sufficient to protect user data.

16. Testing Login and Register Routings

Testing should cover the complete authentication lifecycle. Unit tests can verify password and validation services. Integration tests can verify API routes and database interactions. End-to-end tests can verify a user registering, logging in, accessing the dashboard, logging out, and being denied access afterward. Security tests should verify that credentials are handled safely.

- TC-01: register with valid data.
- TC-02: register with missing identifier.
- TC-03: register with invalid identifier format.
- TC-04: register with weak password.
- TC-05: register with mismatched passwords.
- TC-06: register duplicate account.
- TC-07: login with valid credentials.
- TC-08: login with wrong password.
- TC-09: login with unknown identifier.
- TC-10: access dashboard without login.
- TC-11: access dashboard after login.
- TC-12: access another user's budget.
- TC-13: logout successfully.
- TC-14: access protected route after logout.
- TC-15: password reset with invalid token.
- TC-16: password reset with expired token.
- TC-17: successful password reset.
- TC-18: verification with valid token.
- TC-19: verification with expired token.
- TC-20: repeated login attempts trigger defined protection.
- TC-21: password never appears in application logs.
- TC-22: password hash is stored instead of plaintext.
- TC-23: session/token expiration works.
- TC-24: UI error messages are understandable.
- TC-25: registration-to-login-to-dashboard flow works end-to-end.
- Testing conclusion: every main authentication route should have evidence before final demonstration.

Routing principle: User action → UI route → validation → authentication service → data layer → authenticated state → protected application route. Each step should have defined inputs, outputs, validation, and test evidence.

Security principle: Authentication and authorization must be enforced on the server or trusted application layer; hiding a page in the interface alone is not sufficient to protect user data.

17. Security, Privacy, and Risk Routing

Authentication is security-sensitive. The project should identify risks before implementation and include mitigation measures. The application should use secure transport in production, safe password hashing, protected sessions, authorization checks, input validation, careful logging, dependency management, and security testing. Financial information should be treated as sensitive project data even when the prototype uses sample values.

- Risk: plaintext password storage — mitigation: password hashing.
- Risk: credential interception — mitigation: secure transport in production.
- Risk: account enumeration — mitigation: safe generic responses.
- Risk: brute-force login — mitigation: rate limiting and monitoring.
- Risk: session theft — mitigation: secure session design.
- Risk: unauthorized budget access — mitigation: server-side authorization.
- Risk: insecure password reset — mitigation: short-lived recovery tokens.
- Risk: malicious input — mitigation: validation and sanitization.
- Risk: sensitive logging — mitigation: minimize logs.
- Risk: database exposure — mitigation: access control and secure deployment.
- Risk: dependency vulnerability — mitigation: dependency review.
- Risk: forgotten sessions — mitigation: expiration and logout.
- Risk: incorrect implementation — mitigation: security code review.
- Risk: privacy leakage — mitigation: data minimization.
- Risk: AI exposure of personal data — mitigation: pass only necessary information.
- Risk: demo using real data — mitigation: use fictional sample accounts.
- Risk: weak presentation of security — mitigation: include clear security explanation.
- Privacy should be documented.
- Authentication should be tested continuously.
- Security should be considered throughout development.
- The project should distinguish prototype controls from production controls.
- These controls improve the credibility of the Naan Mudhalvan project.
- Security routing is part of the overall system architecture.
- The final implementation should follow the chosen technology's current security guidance.

Routing principle: User action → UI route → validation → authentication service → data layer → authenticated state → protected application route. Each step should have defined inputs, outputs, validation, and test evidence.

Security principle: Authentication and authorization must be enforced on the server or trusted application layer; hiding a page in the interface alone is not sufficient to protect user data.

18. Documentation and Demonstration Routing

The authentication module should have technical and user documentation. Technical documentation explains routes, request fields, responses, database records, password handling, sessions, and authorization. User documentation explains how to register, log in, use the dashboard, and log out. The live demonstration should use fictional credentials and sample data, never a real password. The routing can be shown visually as Register → Validation → User Service → Database → Login → Authentication → Dashboard → Logout.

- Document Register page.
- Document Login page.
- Document Logout page.
- Document optional recovery page.
- Document optional verification page.
- Document registration API.
- Document login API.
- Document logout API.
- Document session/token approach.
- Document user database fields.
- Document validation rules.
- Document error behavior.
- Document protected routes.
- Document authorization checks.
- Document test cases.
- Prepare fictional demonstration account.
- Open Register page.
- Create sample account.
- Show successful registration.
- Open Login page.
- Log in with sample credentials.
- Show dashboard access.
- Open a budget.
- Show protected budget data.
- Log out.
- Try to access dashboard again.
- Show redirect or unauthorized response.
- Explain password security.
- Explain testing.
- Conclude authentication section before moving to budget functionality.
- This demonstration proves that the login/register routing is connected to the main PocketSmart AI system.

Routing principle: User action → UI route → validation → authentication service → data layer → authenticated state → protected application route. Each step should have defined inputs, outputs, validation, and test evidence.

Security principle: Authentication and authorization must be enforced on the server or trusted application layer; hiding a page in the interface alone is not sufficient to protect user data.

19. Complete Login-to-Budget Routing and Evaluation

The complete routing connects authentication with the core budgeting functionality. A new user registers and creates an account. The account is stored securely. The user logs in, receives authenticated state, and reaches the protected dashboard. The dashboard can then create or open a budget. Income and expenses are processed by the budget services. Logout ends the session. This sequence demonstrates that authentication is not an isolated feature but a foundation for the personalized PocketSmart AI experience.

- Register page → registration request.
- Registration request → validation.
- Validation → duplicate-account check.
- Duplicate check → password hashing.
- Password hash → user database.
- Success → login page or onboarding.
- Login page → login request.
- Login request → user lookup.
- User lookup → password verification.
- Verification success → session/token creation.
- Session/token → protected dashboard.
- Dashboard → authenticated budget list.
- Budget selection → ownership verification.
- Budget → calculation engine.
- Results → dashboard.
- Optional results → AI explanation.
- Logout → session/token termination.
- Post-logout dashboard request → unauthorized response.
- The routing provides a complete lifecycle.
- The project can be evaluated route by route.
- Each route can be linked to test cases.
- Each route can be shown in screenshots.
- Each route can be explained during the Naan Mudhalvan presentation.
- The architecture keeps authentication independent from financial calculations.
- The overall project becomes easier to maintain.
- The complete flow demonstrates practical software engineering.
- The routing can later be expanded for multi-factor authentication and stronger account controls.

Routing principle: User action → UI route → validation → authentication service → data layer → authenticated state → protected application route. Each step should have defined inputs, outputs, validation, and test evidence.

Security principle: Authentication and authorization must be enforced on the server or trusted application layer; hiding a page in the interface alone is not sufficient to protect user data.

20. Final Conclusion and Naan Mudhalvan Submission Checklist

PocketSmart AI: Your Smart Budget, with the subtitle “Login and Register Page Routings,” presents a complete authentication-routing module for the Naan Mudhalvan project. The module provides the secure entry point to personalized budgeting. Registration creates a user identity, login verifies that identity, authentication state protects the dashboard, authorization protects individual budgets, and logout ends access. Optional recovery and verification routes provide a roadmap for future development. The most important conclusion is that authentication should be designed as a separate, testable, secure module that connects cleanly to the rest of the budgeting system.

- Final project title: PocketSmart AI: Your Smart Budget.
- Final document subtitle: Login and Register Page Routings.
- Naan Mudhalvan project context.
- Include project abstract.
- Include objectives.
- Include architecture diagram in the final presentation if available.
- Include Register page screenshot.
- Include Login page screenshot.
- Include dashboard screenshot.
- Include route/API list.
- Include user database design.
- Include password-security explanation.
- Include session/token explanation.
- Include protected-route explanation.
- Include logout explanation.
- Include optional recovery design.
- Include validation rules.
- Include test cases.
- Include test results.
- Use fictional credentials for demonstration.
- Do not place real passwords in screenshots or documentation.
- Show registration.
- Show login.
- Show dashboard access.
- Show budget access.
- Show logout.
- Show blocked access after logout.
- Explain how authentication connects to PocketSmart AI.
- Explain limitations and future improvements.
- Keep source code, screenshots, and documentation consistent.
- Verify all routes before submission.
- Prepare a backup of source code.
- Prepare the final presentation.
- Use clear technical terminology.
- End the presentation by connecting authentication to the smart-budget functionality.
- Final deliverable: a complete, documented, tested login and registration routing module for PocketSmart AI.
- The project should be presented on its actual implemented evidence rather than unsupported claims about ranking or being the “best” project.

Routing principle: User action → UI route → validation → authentication service → data layer → authenticated state → protected application route. Each step should have defined inputs, outputs, validation, and test evidence.

Security principle: Authentication and authorization must be enforced on the server or trusted application layer; hiding a page in the interface alone is not sufficient to protect user data.